

第十章：数据库恢复技术——怎么让"承诺"等于"现实"

根本矛盾

数据库对用户承诺了："我一定把你的数据改对、改对、改对"。

但现实是：硬件会坏、断电、程序会出错、磁头会撞盘。

第十章要解决的核心问题：数据库怎么在出故障后，把"承诺"和"现实"重新对齐？

答案藏在三件事里：事务（最小单位）+ 冗余数据（备份）+ 恢复策略（怎么用备份把数据救回来）。

一、事务的概念与 ACID 4 特性

1.1 什么是事务

事务 = 一组数据库操作序列，要么全做，要么全不做，是不可分割的工作单位。

一个事务可以是一个 SQL 语句、一组 SQL 语句，或者整个程序。一般来说，一个程序包含多个事务。

恍然大悟点："事务" vs "程序"不是一回事！

- 程序 (C/Java) 是代码的整体结构
- 事务 (Transaction) 是数据库操作的逻辑单位
- 一个 main 函数里可以有多个事务（每个事务以 BEGIN TRANSACTION 开始，COMMIT/ROLLBACK 结束）
- 第 8 章的嵌入式 SQL 程序里，整个 main 函数通常就是 1 个事务

事务的两重身份：

- 恢复的基本单位——故障恢复以"事务"为单位，要么整个撤销、要么整个重做
- 并发控制的基本单位——多个事务并发执行时（第 11 章），按"事务"隔离

1.2 ACID 4 特性（必背）

源材料第 281 行明文：原子性、一致性、隔离性、持续性 (Atomicity / Consistency / Isolation / Durability)。

【讲解顺序很重要】——源材料专门强调"一致性和原子性密切相关"。

特性	含义	大白话	银行转账类比
原子性 A	一个事务要么全做，要么全不做	一笔交易只可能 2 种结果——完全成功或完全失败	转账"扣 100 + 加 100"必须同时成功或同时失败
一致性 C	事务执行的结果必须一致	转账前后钱总数不变	A 扣 100 + B 加 100，总和不变
隔离性 I	一个事务的执行不能被其他事务干扰	两笔转账互不干扰	多人同时转账时，不会"看到的余额是错的"
持续性 D (永久性)	事务一旦提交，对数据库的改变应该是永久性的	提交后永不丢失	转账成功后断电也保存

关键逻辑链：一致性靠原子性保证——

源材料第 285 行原文："如果一个事务被迫中断，即事务中的一些操作做了、剩下的还没做，且做了的对



数据库造成的修改已经存入物理数据库的话，就会造成数据库处于不一致的状态”。

推理：如果不允许“部分提交”（原子性）→ 不会出现“部分修改已落盘”→ 也就不会出现不一致（一致性）。所以一致性是原子性的必然结果，二者密切相关。

二、事务的 SQL 定义

3 个 SQL 语句，3 件事：

语句	作用	银行转账类比
BEGIN TRANSACTION	标记事务开始	"开始转账"
COMMIT	提交——事务的所有操作永久生效	"转账成功"
ROLLBACK	回滚——撤销已完成的操作，回到事务开始的状态	"转账失败，恢复原状"

COMMIT 之后事务就结束了；ROLLBACK 之后事务也结束了——两次都意味着“这个事务完事”。

三、4 种故障——“会出什么问题”

数据库可能出 4 种故障，严重程度递增：

故障	别名	性质	例子	损坏范围
事务内部的故障	—	某些预期/非预期错误	运算溢出、违反完整性约束	1 个事务
系统故障	软故障	系统停止运转、需重启	硬件错误、OS 故障、DBMS 代码错误、系统断电	所有未完成事务
介质故障	硬故障	外存物理损坏	磁盘损坏、磁头碰撞、强磁场干扰	数据库本身
计算机病毒	—	恶意程序破坏	—	不定

恍然大悟点：“软故障 vs 硬故障”的关键区别——

- 软故障：内存里数据丢了，但磁盘上的数据库文件没坏（断电就是这种）
- 硬故障：磁盘物理损坏，数据库文件可能也坏了

这种区分直接决定了恢复策略不同（第 5 节细讲）。

四、恢复技术——怎么“未雨绸缪”

4.1 恢复机制全景

源材料第 302-304 行明文：

恢复机制涉及两个关键问题：

1. 如何建立冗余数据（备份）
2. 如何利用这些冗余数据实施数据库的恢复

>



恢复的基本原理：利用存储在后备副本、日志文件和数据库镜像中的冗余数据来重建数据库。

三种冗余数据源：

冗余数据	是什么	谁负责建立	用于
后备副本	数据库的"备份文件"	数据转储	介质故障恢复
日志文件	事务操作的"流水账"	登记日志	事务/系统故障恢复
数据库镜像	数据库的"实时复制"	DBMS 自动	介质故障预防

4、5、6 节就是分别讲这三种机制。本节先讲前两种（转储 + 日志），第 5 节讲数据库镜像。

4.2 数据转储

数据转储 = DBA 定期把整个数据库复制到磁带/磁盘上保存。复制出来的数据叫后备副本（或后援副本）。

两个维度的分类：

维度	静态转储	动态转储
转储时是否允许事务运行	不允许（无运行事务）	允许（边备份边用）
优点	数据一致性好	不影响业务
缺点	必须停服	转储期间数据库还在变，后备副本可能不一致

维度	海量转储	增量转储
每次转储什么	全部数据库	只转上次转储后更新过的数据
优点	恢复简单	速度快、空间小
缺点	慢、占空间	恢复时要按顺序合并多次增量

2×2 组合 = 4 种转储方法（这是高频考点）：

组合	特点
动态海量转储	边跑边全部备份
动态增量转储	边跑边增量备份
静态海量转储	停服全部备份
静态增量转储	停服增量备份

4.3 登记日志文件

日志文件 = 记录事务对数据库的更新操作的文件（"流水账"）。

两种格式：

以记录为单位的日志文件需要登记的内容包括：各个事务的开始标记（BEGIN TRANSACTION）、各个事务的结束标记（COMMIT/ROLLBACK）、各个事务的所有更新操作。

每条日志记录的具体内容：

格式	记录内容	类比
以记录为单位	事务标识、操作类型、操作对象、更新前旧值、更新后新值	银行流水的"每笔交易详情"
以数据块为单位	事务标识、被更新的数据块	只记"今天动了哪些账本"

插入时"旧值"为空；删除时"新值"为空。

日志文件的 3 条作用（高频考点，必背）

- 1. 事务故障恢复和系统故障恢复必须用日志文件——没日志救不回来
- 2. 动态转储方式中必须建立日志文件——因为动态转储期间数据库还在变，光靠后备副本不准确，必须后备副本 + 日志结合
- 3. 静态转储方式中也可以建立日志文件——装入后备副本后，配合日志做重做/撤销，能恢复到故障点
恍然大悟点："必须用" vs "也可以用"的区别在于"动态转储期间数据库还在变"——这种情况下后备副本是过时的，必须靠日志补齐。

登记日志的两条原则（必背）

- 1. 登记的次序严格按并发事务执行的时间次序——多个事务并发时，日志不能乱序，否则恢复时因果错乱
- 2. 必须先写日志文件，后写数据库——核心原则（用会计记账比喻：先记账，再动钱）
为什么"先写日志"反了会怎样？
 比喻：银行出纳——"先在账本上记一笔（写日志），再把钱给客户（写数据库）"。
 >
 如果反了："先把钱给客户，再记账"——万一中间断电，账本上没记录，但钱已经给出去了——钱去哪儿了不知道。
 >
 "先写日志"保证：即使写数据库之前断电，日志里已经记了"要改什么"——恢复时按日志来改，保证不丢。

五、不同故障的恢复策略——"具体怎么救"

4 种故障对应不同恢复策略（这是源材料的核心考点）：

故障	恢复策略	谁来执行	用什么
事务故障	UNDO（撤销）此事务已做的修改	系统自动完成，对用户透明	日志文件
系统故障（软）	UNDO（撤销）未完成的事务 + REDO（重做）已完成的事务	系统重启时自动完成，不需要用户干预	日志文件
介质故障（硬）	重装数据库 + REDO（重做）已完成的事务	DBA 手动 + 系统配合	后备副本 + 日志文件
计算机病毒	同介质故障（最严重）	视破坏程度	后备副本 + 日志

UNDO vs REDO 怎么区分？（高频易混点）

操作	含义	适用场景
UNDO（撤销）	把事务已经做的修改改回去	事务没完成（未提交就被打断）
REDO（重做）	把事务已经做的修改再做一遍	事务已提交，但修改可能没落盘（断电导致）

速记口诀：



- 事务没干完 → 撤销 (UNDO)
- 事务已提交但数据没落盘 → 重做 (REDO)

5.4 数据库镜像——避免介质故障的预防方案

源材料第 339 行（带 * 号）：为避免磁盘介质出现故障影响数据库，许多 DBMS 提供了数据库镜像功能（复制数据实现）。

关键区分：

方案	性质	触发时机
数据转储	事后补救	故障发生后用后备副本恢复
日志文件	辅助补救	配合后备副本，事后追到故障点
数据库镜像	事中预防	故障发生前实时复制数据到另一磁盘

数据库镜像 = 实时把数据复制到另一块磁盘。一旦主磁盘坏了，备用磁盘上的数据是最新的，立即接管。

六、检查点恢复技术——"效率优化"

6.1 问题：恢复的"代价"

如果恢复时对所有事务都 REDO，日志文件很长时非常慢。能不能只对必要的事务 REDO?

6.2 解决方案：检查点

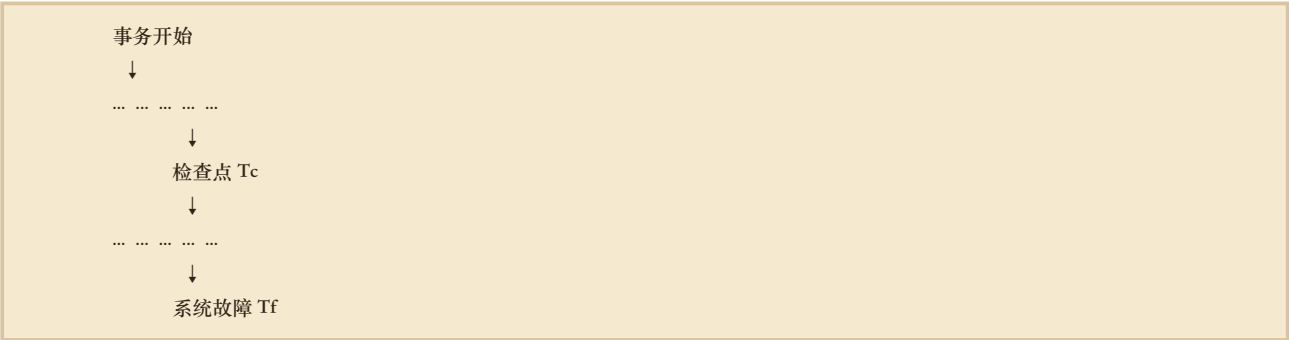
检查点 = 在日志文件中打个标记，标记之前的日志可以"安全丢弃"——因为那些事务的修改已经写入磁盘了。

源材料原文："检查点之前的事务已经做完了，不需要重做，检查点期间和之后的事务需要重做"

检查点 = 恢复的"快捷键"——跳过无需 REDO 的部分，只处理检查点之后的。

6.3 故障时不同事务的 4 种状态

设检查点时刻为 T_c ，故障发生时刻为 T_f 。事务有 4 种可能状态（用 T_1 、 T_2 、 T_3 、 T_4 表示）：



事务	状态	恢复策略
T_1	在检查点之前开始并完成	不要重做（修改早已落盘）
T_2	在检查点之前开始，检查点之后完成	重做 (REDO)
T_3	在检查点之前开始，故障时还没完成	撤销 (UNDO)
T_4	在检查点之后开始，故障时还没完成	撤销 (UNDO)



速记口诀:

- 检查点之前完成的事务 → 不重做
- 检查点之后完成的事务 → 重做
- 故障时未完成的事务 → 撤销

七、跨章关联

第十章是"恢复", 与前后章节的连接:

关联章节	关联点
第 7 章 数据库设计	维护阶段"转储和恢复" = 本章内容
第 8 章 数据库编程	COMMIT/ROLLBACK = 事务的提交/回滚; 嵌入式 SQL 程序 = 一个或多个事务
第 11 章 并发控制	隔离性的实现 = 第 11 章内容 (用封锁/时间戳/快照隔离)

OCR 错误处理

源材料第 5-9 页有少量 OCR 错误, 订正如下:

源材料原文 (错误)	应该是
第 280/333/335 行 "撤销"	撤销
第 317 行 "I 日值"	旧值 (OCR 将"旧"识别为"I 日")
第 335 行 "未完成"	未完成
第 335 行 "童重新启动"	重新启动 (OCR 将"重"识别为"童")
第 344 行 "T. (檢 点)" (繁体+错字)	T (检查点)
第 351-354 行 T1-T5 状态散乱	已按"4 种状态"重新组织

易混淆对比总结

表 1 : ACID 4 特性

特性	含义	一句话
原子性 A	全做或全不做	不可分割
一致性 C	转账前后总和不变	靠原子性保证 (密切相关)
隔离性 I	多事务互不干扰	第 11 章并发
持续性 D	提交后永久不变	断电也不丢

表 2 : 4 种故障

故障	别名	严重度	恢复策略
事务内部	—	轻	UNDO
系统	软故障	中	UNDO + REDO

介质	硬故障	重	重装 + REDO
计算机病毒	—	重	同介质

表 3：数据转储的 4 种组合

组合	是否停服	是否全量
动态海量	否	全量
动态增量	否	增量
静态海量	是	全量
静态增量	是	增量

表 4：日志文件的 2 种格式

格式	记录内容	类比
以记录为单位	事务/操作类型/对象/旧值/新值	银行流水每笔详情
以数据块为单位	事务/被更新的数据块	只记"动了哪些账本"

表 5：UNDO vs REDO

操作	含义	适用
UNDO（撤销）	把已做的修改改回去	事务没完成
REDO（重做）	把已做的修改再做一遍	事务已提交但没落盘

表 6：检查点恢复的 4 种事务状态

状态	例子	恢复策略
检查点前完成	T1	不重做
检查点后完成	T2	重做
故障时未完成（检查点前开始）	T3	撤销
故障时未完成（检查点后开始）	T4	撤销

表 7：三种冗余数据机制对比

机制	性质	触发时机	适用故障
后备副本（数据转储）	事后补救	故障发生后用	介质故障
日志文件	事后追到故障点	配合后备副本/单独用	事务/系统故障
数据库镜像	事中预防	故障发生前实时复制	介质故障

逻辑主线

数据库对用户"承诺" vs 现实可能"故障"



[核心概念] 事务 —— 不可分割的工作单位

- ├— 一个程序包含多个事务
- ├— ACID 4 特性（原子性→一致性→隔离性→持续性）
- └— SQL 定义：BEGIN TRANSACTION / COMMIT / ROLLBACK



[会出什么错] 4 种故障

- ├— 事务内部（运算溢出、违反约束）
- ├— 系统软故障（断电、OS 故障）
- ├— 介质硬故障（磁盘损坏、磁头撞）
- └— 计算机病毒



[怎么预防和补救] 恢复机制

- ├— 恢复机制全景：2 个关键问题 + 3 种冗余数据源
- |
- ├— 机制一：数据转储
- | ├— 静态 vs 动态（是否停服）
- | ├— 海量 vs 增量（是否全量）
- | └— 4 种组合
- |
- ├— 机制二：登记日志文件
- | ├— 2 种格式（记录/数据块）
- | ├— 3 条作用（事务/系统恢复必须用 / 动态转储必须建 / 静态转储也能建）
- | └— 2 条原则（按时间次序 / 先写日志后写数据库）
- |
- └— 机制三：数据库镜像（事中预防 vs 事后补救）



[具体怎么救] 不同故障的恢复策略

- ├— 事务故障 → UNDO
- ├— 系统故障 → UNDO + REDO
- └— 介质故障 → 重装 + REDO



[效率优化] 检查点恢复

- ├— 跳过无需 REDO 的事务



跨章呼应：第 8 章 COMMIT/ROLLBACK / 第 11 章隔离性

整个第十章就一件事：在故障不可避免的前提下，用"事务 + 冗余数据 + 恢复策略"让数据库重新对齐"承诺"和"现实"。

